MCP · HANDS-ON TRAINING

Building with MCP

From your first server to a secure custom agent



What you will be able to do



Explain MCP

Hosts, clients, servers, and the protocol between them



Build a server

Expose your own tools to any AI app



Connect any agent

Claude Desktop, Cursor, or your own chatbot



Secure it

OAuth 2.1 authorization for remote servers

The N×M integration problem

BEFORE: CUSTOM CONNECTORS

$$2 \times 4 = 8$$

2 models × 4 systems

10 × 50 = 500

WITH MCP: ONE PROTOCOL

$$2 + 4 = 6$$

2 clients + 4 servers

10 + 50 = 60

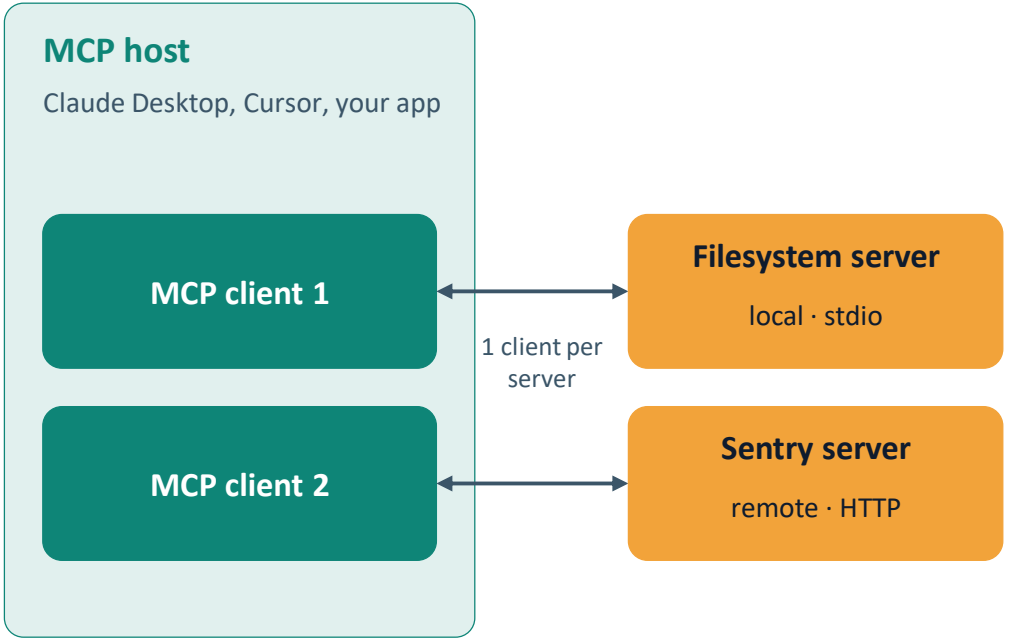
Each model implements one MCP client. Each system exposes one MCP server.

What is MCP?

An open protocol that connects LLM applications to external tools and data.

Build a server once and every MCP host can use it.

It standardizes context exchange only. How you call the LLM stays up to you.



Two layers, two transports

Transport layer

stdio · Streamable HTTP · authorization

Data layer

JSON-RPC 2.0 messages

Discovery, tools, resources, prompts, and notifications

	stdio	Streamable HTTP
Runs	Local process	Remote server
Serves	Usually one client	Usually many clients
Auth	Env credentials	Bearer token or OAuth

Both transports carry the same JSON-RPC 2.0 messages, so your tools work the same either way.

Three server primitives



Tools

Functions the AI app can call to take action

e.g. File operations, API calls, database queries



Resources

Data that gives the AI app context

e.g. File contents, database records, API responses



Prompts

Reusable templates that structure interactions

e.g. System prompts, few-shot examples

Discover with */list · Fetch with */get · Run tools with tools/call

One request, step by step

1 Discover

`server/discover`

Server returns its versions and capabilities

2 List tools

`tools/list`

Names, descriptions, and JSON Schema inputs

3 Call a tool

`tools/call`

Send the name and arguments, get content back

4 Get notified

`list_changed`

Opt in with subscriptions/listen, then list again

Spec 2026-07-28 is stateless: every request carries its protocol version and capabilities. Older clients still open with an initialize handshake.

Build your first server

```python
from mcp.server import MCPServer

mcp = MCPServer("weather")

@mcp.tool()
async def get_alerts(state: str) -> str:
    """Get weather alerts for a US state.

    Args:
        state: Two-letter code (e.g. CA)
    """
    ...  # call your API, return text

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Type hints and the docstring become the tool schema.

With stdio, the host launches your server as a subprocess.

Needs Python SDK 2.0 or newer. TypeScript uses the server package with zod.

Design tools models can use

DO

- Use clear, unique names such as calculator_arithmetic
- Describe what the tool does and when to use it
- Return failures in the result so the model can retry
- Log to stderr or a file in stdio servers

DON'T

- Print to stdout in a stdio server: it corrupts the JSON-RPC stream
- Use vague names like calculate
- Pack everything into one giant tool
- Trust model-supplied arguments without validating them

Test it, then connect it

1. Test with the MCP Inspector

```
# interactive web UI
npx @modelcontextprotocol/inspector

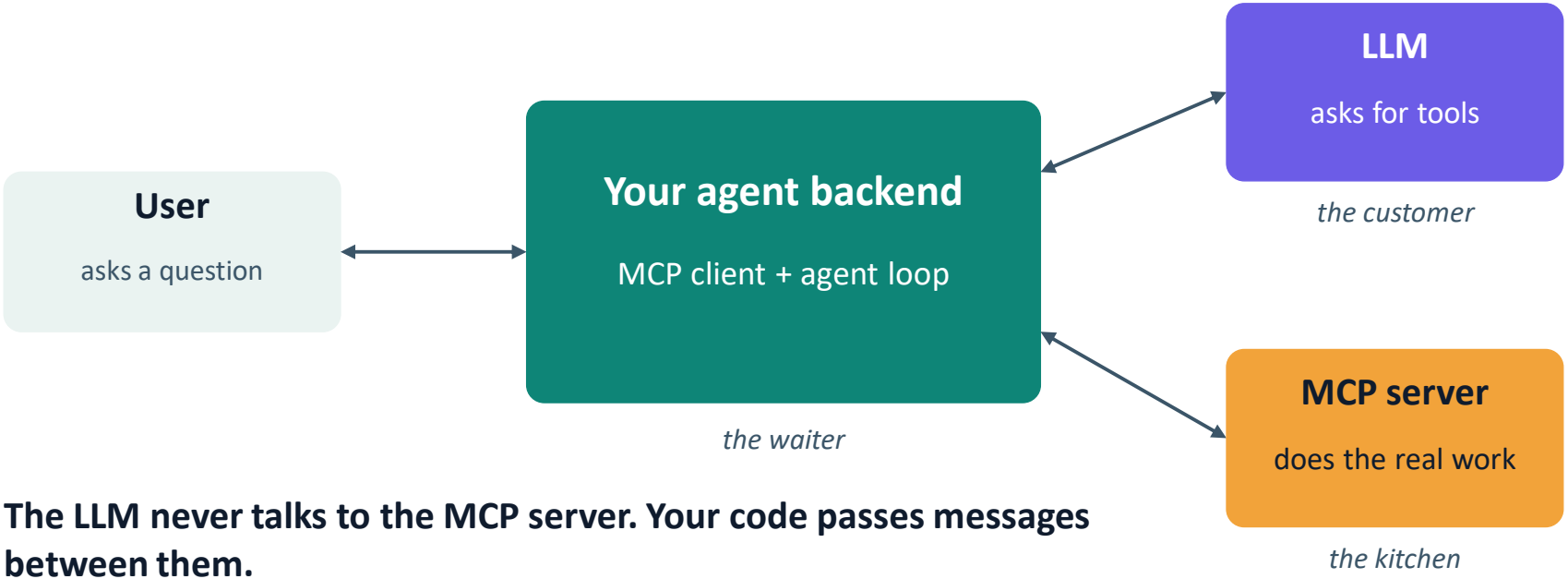
# or scripted (CLI)
npx @modelcontextprotocol/inspector --cli \
  node build/index.js \
  --method tools/list
```

2. Register it in a host

```
{
  "mcpServers": {
    "weather": {
      "command": "uv",
      "args": ["--directory", "/ABS/PATH",
               "run", "weather.py"]
    }
  }
}
```

Use absolute paths, and fully quit and restart the app after editing the config. Server logs appear in the host's MCP log files.

Custom agent = your own host



The LLM never talks to the MCP server. Your code passes messages between them.

The agent loop in code

1. Connect and list the server's tools
2. Convert schemas to your LLM's tool format
3. Send the question and tools to the LLM
4. If it asks for a tool, run call_tool
5. Return the result, repeat until it answers

```
# params = StdioServerParameters(...)
async with Client(stdio_client(params)) as c:
    tools = (await c.list_tools()).tools
    llm_tools = [{
        "name": t.name,
        "description": t.description,
        "input_schema": t.input_schema,
    } for t in tools]
    # LLM asks for a tool, so run it:
    r = await c.call_tool(name, args)
    # send r.content back to the LLM;
    # r.is_error lets the model retry
```

Python SDK 2.0 shown. SDK 1.x used ClientSession and initialize().

Configure servers per user

	Desktop hosts	Your custom agent
Config	claude_desktop_config.json	Your DB or YAML, per user or tenant
Transport	Mostly stdio	Streamable HTTP for web chatbots
Credentials	Env vars in the config file	Per-user tokens, stored encrypted
Tool access	All of the server's tools	You choose what each user sees
Consent	Host shows approval prompts	You build approvals for risky tools

Do you need authorization?

Add it when…

- Your server reads user-specific data
- You must audit who did what
- Actions need user consent
- Enterprise access controls apply
- You want per-user rate limits



Local server (stdio)

Use environment credentials, or credentials from a library inside the server.



Remote server (HTTP)

Use OAuth 2.1: the client gets a token and the server validates it.

Authentication and authorization

Who?

Authentication

Who are you?

Proves the identity of the client or user, such as an API key or an OAuth sign-in.

What?

Authorization

What can you do?

Decides which tools and data that identity can use, controlled with scopes.

Why it matters

Real access. Servers reach databases, files and APIs.

Least privilege. A leaked token or a wrong tool call does less harm.

Safe sharing. Per-user access makes remote servers safe to deploy.

Authenticate first, then authorize every tool call.

Hands-on labs

1	**Weather server**	Follow the docs quickstart and test with the Inspector. The NWS API is US-only, so swap in a data source you can use locally.
2	**Connect a host**	Register it in Claude Desktop or Cursor and call a tool from chat.
3	**Custom agent**	Build the agent loop: list tools, call the LLM, run its tool requests.
4	**Add authorization**	Run Keycloak in Docker and protect your server with OAuth.

Takeaways and resources

KEY TAKEAWAYS

1. MCP turns N×M integrations into N+M
2. Servers expose tools; hosts connect through clients
3. Your agent loop sits between the LLM and the server
4. Remote servers need OAuth: validate every token

RESOURCES

modelcontextprotocol.io
Docs and specification

MCP Inspector
Test and debug servers

Official SDKs
Python, TypeScript, and more

Reference servers
github.com/modelcontextprotocol/servers

MCP Registry
Publish your server

Source: modelcontextprotocol.io docs, spec version 2026-07-28

Your task

1	**Pick a use case**	An AI agent, a chatbot, or any other AI application you want to extend.
2	**Choose resources**	A database, web search, GitHub, files, or similar tools and data.
3	**Build server + client**	An MCP server for those resources, plus your own user-defined MCP client, not a built-in one.
4	**Integrate and demo**	Connect the server's tools and resources to your AI app and show it working.